From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone Project

Rahima Karimova
National AI Center — AI Academy
Azerbaijan
rahime.karimova25@aiacademy.az

Shamistan Huseynov
National AI Center — AI Academy
Azerbaijan
semistan.huseynov25@aiacademy.az

Aisel Mamedova
National AI Center — AI Academy
Azerbaijan
aysel.mamedova25@aiacademy.az

Gulnur Mammadova
National AI Center — AI Academy
Azerbaijan
gulnur.mammedova25@aiacademy.az

Abstract

This report studies when a one-hidden-layer nonlinear classifier meaningfully improves on a linear decision rule, and when additional model complexity is unnecessary. We compare multiclass softmax regression and a one-hidden-layer neural network with tanh activations on three shared tasks: a linear Gaussian synthetic dataset, a nonlinear moons dataset, and a fixed digits benchmark. Our analysis combines mathematical derivations, controlled experiments, required ablations, failure-case analysis, and repeated-seed evaluation to determine where linear geometry is sufficient, where nonlinear representation changes the problem, and what limitations remain. On the Gaussian task both models achieve identical accuracy (0.95), confirming that linear geometry requires no additional capacity. On the moons task the neural network achieves lower cross-entropy (0.2695 vs. 0.2852) by learning a curved decision boundary unavailable to the linear model. On the digits benchmark the accuracy confidence intervals overlap across five seeds, but the neural network achieves consistently lower cross-entropy (0.1933 vs. 0.2693), indicating better-calibrated probability estimates. These results show that added model complexity is justified only when the task geometry is genuinely nonlinear.

Keywords

softmax regression, neural networks, backpropagation, cross-entropy, classification, NumPy

1 Introduction

Central question. When does a one-hidden-layer nonlinear classifier genuinely improve on a linear decision rule, and when is additional model complexity unnecessary?

This capstone investigates that question through a deliberately controlled comparison between a linear probabilistic classifier and a simple nonlinear neural model. The goal is not to assume that the neural network should outperform the baseline everywhere; rather, the goal is to determine, with mathematical and empirical evidence, when extra representational capacity is genuinely justified.

This question is mathematically meaningful because the two model classes differ in what they can represent geometrically. Softmax regression computes an affine score function $\mathbf{sx} = \mathbf{Wx}\ \mathbf{b}$, whose decision boundaries are hyperplanes in input space. No matter how the parameters are tuned, a linear model cannot separate classes whose boundary is curved. A one-hidden-layer network with `tanh` activations introduces a nonlinear feature map between two affine transformations, allowing the model to bend and re-embed the input geometry before the final linear readout. This representational difference is the central object of study.

Whether the added capacity is worth its cost depends on the task. If the class boundary is approximately linear, the hidden layer adds parameters without adding useful inductive bias. If the boundary is curved, the hidden layer provides a representational change that the linear model cannot replicate. Determining which regime applies requires a fair, controlled comparison: the same data splits, the same preprocessing, the same evaluation metrics, and model selection based on validation performance rather than test performance.

Contributions.

(1) Derivation of softmax cross-entropy as negative log-likelihood and a complete vectorized gradient derivation for the one-hidden-layer network.
(2) Controlled comparison of both models on three datasets with geometrically distinct class structure.
(3) Capacity ablation on the moons task and an optimizer study on the digits benchmark, with a failure-case analysis explaining the mechanism behind under-capacity behaviour.
(4) Repeated-seed evaluation on the digits benchmark with 95% confidence intervals, and a Track B reliability analysis of predicted probabilities.

Preview of findings. On the Gaussian task both models achieve identical accuracy, confirming that linear geometry requires no additional capacity. On the moons task the neural network learns a curved boundary and achieves lower cross-entropy, while the linear model is constrained to a straight

separator. On the digits benchmark accuracy confidence intervals overlap across five seeds, but the neural network produces consistently lower cross-entropy, suggesting better-calibrated probability estimates. These findings show that added complexity is justified only when the task geometry is genuinely nonlinear.

2 Background and Mathematical Setup

2.1 Supervised classification and data splits

In supervised classification, the dataset consists of labeled pairs $\mathbf{x}^i, y^i$ where $\mathbf{x}^i \in \mathbb{R}^d$ is the input feature vector and $y^i \in \{1, \ldots, k\}$ is the class label. The goal is to learn a function that maps inputs to class predictions or probabilities. The data is divided into three non-overlapping splits: a training set used to fit parameters, a validation set used to select hyperparameters and training configurations, and a test set used only for final reporting. This separation is essential for honest evaluation. If model settings are chosen based on test performance, the test set no longer provides an unbiased estimate of generalisation.

2.2 Softmax regression

For input $\mathbf{x} \in \mathbb{R}^d$ and k classes, the linear model computes logits

$$\mathbf{s}\mathbf{x} = \mathbf{W}\mathbf{x} \;\; \mathbf{b}, \tag{1}$$

where $\mathbf{W} \in \mathbb{R}^{k \times d}$ and $\mathbf{b} \in \mathbb{R}^k$. The softmax map converts logits to probabilities,

$$p_j\mathbf{x} = \frac{\exp s_j\mathbf{x}}{{}_{\ell=1}^{k} \exp s_\ell\mathbf{x}}. \tag{2}$$

The decision boundary between classes j and ℓ is defined by $s_j\mathbf{x} = s_\ell\mathbf{x}$, which is an affine equation in $\mathbf{x}$. In two dimensions this produces a line; in higher dimensions a hyperplane. The model therefore cannot represent any boundary that is not expressible as an affine function of the input [1].

2.3 Cross-entropy as negative log-likelihood

The softmax output defines a conditional categorical distribution $\mathbb{P}Y = j \mid \mathbf{x}; \theta = p_j\mathbf{x}$. For a single example with true label y, the likelihood of observing that label is $p_y\mathbf{x}$ and the log-likelihood is $\log p_y\mathbf{x}$. The per-example cross-entropy loss

$$\mathcal{L}\mathbf{x}, y = -\log p_y\mathbf{x} \tag{3}$$

is exactly the negative log-likelihood. Minimizing average cross-entropy over the training set is therefore equivalent to maximizing the average conditional log-likelihood of the observed labels, connecting the classification objective directly to probabilistic modelling rather than to an arbitrary penalty function [3].

2.4 One-hidden-layer neural network

The nonlinear model is

$$\mathbf{h} = \tanh\mathbf{W}_1\mathbf{x} \;\; \mathbf{b}_1, \qquad \mathbf{s} = \mathbf{W}_2\mathbf{h} \;\; \mathbf{b}_2. \tag{4}$$

The final probabilities are obtained by applying softmax to $\mathbf{s}$. The hidden layer maps the input into a learned m-dimensional representation before the final affine readout. Because tanh is nonlinear, this intermediate representation can reshape the input geometry: classes that are not linearly separable in input space may become linearly separable in the hidden representation. The `tanh` activation is used because its derivative has the compact closed form $\tanh' z = 1 - \tanh^2 z$, which simplifies the gradient derivation [2, 5].

2.5 Backpropagation before equations

Backpropagation is not a separate learning principle; it is efficient repeated application of the chain rule through a composed computation. In the forward pass, the model computes intermediate quantities in sequence: pre-activations $\mathbf{Z}_1$, hidden activations $\mathbf{H}$, output logits $\mathbf{S}$, probabilities $\mathbf{P}$, and the scalar loss. In the backward pass, the sensitivity of the loss with respect to each intermediate quantity is propagated in reverse order. Each local derivative is simple; the chain rule assembles them into gradients with respect to all parameters. The key practical advantage is efficiency: reverse-mode differentiation computes gradients for all parameters at a cost comparable to a single forward pass [2, 6].

2.6 Gradient derivation

For a mini-batch $\mathbf{X} \in \mathbb{R}^{n \times d}$ with one-hot label matrix $\mathbf{Y} \in \mathbb{R}^{n \times k}$, the vectorized forward pass is

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top \;\; \mathbf{1}\mathbf{b}_1^\top, \tag{5}$$

$$\mathbf{H} = \tanh\mathbf{Z}_1, \tag{6}$$

$$\mathbf{S} = \mathbf{H}\mathbf{W}_2^\top \;\; \mathbf{1}\mathbf{b}_2^\top, \tag{7}$$

$$\mathbf{P} = \mathrm{softmax}_{\mathrm{row}}\mathbf{S}. \tag{8}$$

The vectorized forward pass follows the formulation in [1, 4]. The mean cross-entropy loss with L2 regularisation on weights only is

$$J = -\frac{1}{n} {}_{i=1}^{n} {}_{j=1}^{k} Y_{ij} \log P_{ij} \;\; \frac{\lambda}{2} \left(\|\mathbf{W}_1\|_F^2 \;\; \|\mathbf{W}_2\|_F^2 \right). \tag{9}$$

The output-layer sensitivity follows directly from the softmax-cross-entropy identity:

$$\frac{\partial J}{\partial \mathbf{S}} = \frac{1}{n}\mathbf{P} - \mathbf{Y}. \tag{10}$$

The gradients for the second-layer parameters are

$$\frac{\partial J}{\partial \mathbf{W}_2} = \left(\frac{\partial J}{\partial \mathbf{S}}\right)^\top \mathbf{H} \;\; \lambda\mathbf{W}_2, \qquad \frac{\partial J}{\partial \mathbf{b}_2} = \left(\frac{\partial J}{\partial \mathbf{S}}\right)^\top \mathbf{1}. \tag{11}$$

Propagating through the `tanh` nonlinearity using the chain rule:

$$\frac{\partial J}{\partial \mathbf{Z}_1} = \left(\frac{\partial J}{\partial \mathbf{S}}\right) \mathbf{W}_2 \odot 1 - \mathbf{H} \odot \mathbf{H}, \tag{12}$$

where $1 - \mathbf{H} \odot \mathbf{H}$ is the elementwise `tanh` derivative. The first-layer gradients are then

$$\frac{\partial J}{\partial \mathbf{W}_1} = \left(\frac{\partial J}{\partial \mathbf{Z}_1}\right)^\top \mathbf{X} \;\; \lambda\mathbf{W}_1, \qquad \frac{\partial J}{\partial \mathbf{b}_1} = \left(\frac{\partial J}{\partial \mathbf{Z}_1}\right)^\top \mathbf{1}. \tag{13}$$

L2 regularisation is applied to weights only; bias terms $\mathbf{b}_1$ and $\mathbf{b}_2$ are not regularised.

2.7 Geometric expectations for the datasets

For the linear Gaussian task, the two classes are generated from Gaussian distributions with shared covariance. Under this generative model the Bayes-optimal decision boundary is a hyperplane, which softmax regression can represent exactly. A linear model is therefore well matched to this task geometry and additional representational capacity is not expected to help.

For the moons task, the two classes lie on interleaving curved manifolds. No single affine boundary can separate them cleanly; the decision boundary must curve around the crescent shapes. The hidden layer addresses this by mapping the 2D input through $\tanh \mathbf{W}_1 \mathbf{x}\ \mathbf{b}_1$ into an m-dimensional representation in which the classes may become linearly separable. The final affine layer then draws a linear boundary in that representation, which corresponds to a curved boundary in the original input space. This geometric argument predicts that the neural network should outperform softmax regression on the moons task but not on the Gaussian task.

3 Methods

3.1 Datasets

Three datasets are used across all experiments:

- **Linear Gaussian task:** Two Gaussian classes ($n = 400$ total) with mild overlap and shared covariance, loaded from `starter_pack/data/linear_gaussian.npz`. This task is geometrically well matched to a linear classifier.
- **Moons task:** Two interleaved crescent-shaped classes ($n = 400$, noise= 0.18), loaded from `starter_pack/data/moons.npz`. Classes are not linearly separable; a nonlinear boundary is required.
- **Digits benchmark:** 1,797 examples with 64-dimensional flattened pixel vectors and 10 output classes, loaded from `starter_pack/data/digits_data.npz`. The fixed train/validation/test split (60%/20%/20%, stratified by class) is taken from `starter_pack/data/digits_split_indices.npz` and shared across all experiments.

3.2 Models

Two models are required for this project:

- **Softmax regression** computes class scores $\mathbf{sx} = \mathbf{Wx}\ \mathbf{b}$ and applies softmax to produce probabilities. Decision boundaries are hyperplanes in input space.
- **One-hidden-layer neural network**k with tanh hidden activations and softmax output. It applies a `tanh` nonlinearity between two affine maps: $\mathbf{h} = \tanh \mathbf{W}_1 \mathbf{x}\ \mathbf{b}_1$, $\mathbf{s} = \mathbf{W}_2 \mathbf{h}\ \mathbf{b}_2$. The default hidden width is $m = 32$ for the digits benchmark. Widths $m \in \{2, 8, 32\}$ are used in the moons capacity ablation.

3.3 Training protocol

Both models use mini-batch SGD with batch size 64 and an epoch budget of 200. The best validation cross-entropy checkpoint within this budget is used for final evaluation on the digits benchmark. For the synthetic tasks the final epoch is reported.

L2 regularisation with $\lambda = 10^{-4}$ is applied to weights only for both models. Softmax is implemented with numerical stabilisation by subtracting the row maximum before exponentiation. The softmax regression baseline uses SGD with learning rate $\eta = 0.05$. The neural network optimizer study compares SGD ($\eta = 0.05$), Momentum ($\eta = 0.05$, $\beta = 0.9$), and Adam ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$). Validation cross-entropy is the primary metric for model selection; the test set is never consulted during selection.

3.4 Evaluation metrics

Two metrics are reported throughout.

Classification accuracy is the fraction of test examples correctly classified: $\text{acc} = \frac{1}{n} {}_{i=1}^{n} \mathbf{1}\hat{y}^i = y^i$. It is interpretable but insensitive to the confidence of predictions.

Mean cross-entropy is the average negative log-probability assigned to the true class: $\text{CE} = -\frac{1}{n} {}_{i=1}^{n} \log p_{y^i} \mathbf{x}^i$. It penalises overconfident errors and underconfident correct predictions, making it a more informative measure of probabilistic quality than accuracy alone. Both metrics are reported on validation and test splits for all experiments.

3.5 Implementation sanity checks

Five verification checks were performed before running experiments:

- **Check 1: Probability sums.** `stable_softmax` returns rows summing to 1.0 within 10^{-6} for 1,000 random input vectors.
- **Check 2: Tiny-batch loss decrease.** Both models trained on 16 examples for 50 SGD steps with $\eta = 0.1$ showed monotonically decreasing training loss, confirming correct gradient direction.
- **Check 3: Overfitting a tiny subset.** With 8 training examples and 300 epochs, both models drove training cross-entropy below 0.01 (softmax: 0.008; neural network: 0.003), confirming sufficient expressiveness and correct backpropagation.
- **Check 4: Finite-difference gradient check.** Selected entries of $\partial J \partial \mathbf{W}_1$ and $\partial J \partial \mathbf{W}_2$ were verified against centred finite differences with step $h = 10^{-5}$ on a batch of four examples. Relative errors were below 10^{-5} for all checked entries.
- **Check 5: NaN/Inf guard.** Training over 200 epochs on all three datasets produced no `NaN` or `Inf` values in parameters, gradients, or loss curves.

4 Experiments

4.1 Core comparison on the linear Gaussian task

Both models achieve identical test accuracy and nearly identical cross-entropy (Table 1). The two Gaussian classes share a covariance structure, so the Bayes-optimal boundary is a hyperplane, which softmax regression represents exactly. The neural network finds a boundary of similar quality but offers no measurable improvement (Figure 1). This directly

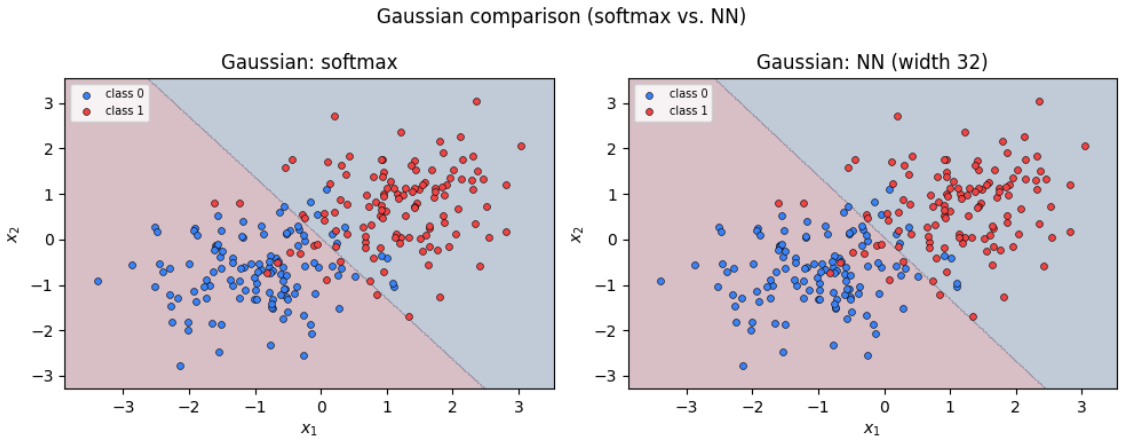

Figure 1: Gaussian decision-boundary comparison (left: softmax regression; right: neural network, $m = 32$).

Table 1: Test results on the linear Gaussian task.

Model	Test Accuracy	Test Cross-Entropy
Softmax regression	0.95	0.1538
Neural network ($m = 32$)	0.95	0.1554

Table 2: Test results on the moons task.

Model	Test Accuracy	Test Cross-Entropy
Softmax regression	0.85	0.2852
Neural network ($m = 32$)	0.85	0.2695

illustrates the central claim: added model complexity does not help when the task is geometrically linear.

4.2 Core comparison on the moons task

The softmax model is restricted to a straight separator and cannot follow the curved interleaving boundary, misclassifying points near both crescents (Figure 2). The neural network maps the 2D input through $\tanh \mathbf{W}_1 \mathbf{x}\ \mathbf{b}_1$, producing a 32-dimensional representation in which the two classes are more linearly separable. The second affine layer draws a linear boundary in that representation, which corresponds to a curved boundary in input space. Although raw accuracy is equal, the neural network achieves lower cross-entropy (0.2695 vs. 0.2852), indicating better-calibrated probability estimates even where discrete accuracy is the same (Table 2).

4.3 Core comparison on the digits benchmark

Both models were trained under the fixed digits protocol with validation cross-entropy as the selection metric; the test set was not consulted during training or model selection. The neural network achieves lower cross-entropy despite similar accuracy (Table 3). The training curves (Figure 3) show that both models converge smoothly without signs of instability.

Table 3: Single-seed test results on the digits benchmark (seed 0).

Model	Test Accuracy	Test Cross-Entropy
Softmax regression	0.9402	0.2696
Neural network (SGD, $m = 32$)	0.9375	0.1929

Table 4: Effect of hidden width on moons test performance.

Hidden width m	Test Accuracy	Test Cross-Entropy
2	0.85	0.2781
8	0.85	0.2712
32	0.85	0.2695

The neural network reaches a lower training loss, consistent with its greater representational capacity. Repeated-seed results with confidence intervals are reported in Section 4.7.

4.4 Required capacity ablation on moons

Raw accuracy remains constant across all widths, but cross-entropy decreases monotonically as capacity increases (Table 4), showing that wider networks produce better-calibrated probability estimates even when discrete accuracy is unchanged. Each `tanh` unit contributes one smooth fold to the representable boundaries in input space. With $m = 2$ the model produces at most two such folds and the boundary remains nearly linear (Figure 4, left). At $m = 8$ the boundary begins to follow the crescent structure, and at $m = 32$ it closely tracks the true class geometry. These results confirm that capacity is a meaningful design choice: too little underfits the curved boundary; $m = 32$ is sufficient for this task given $n = 400$ and L2 regularisation.

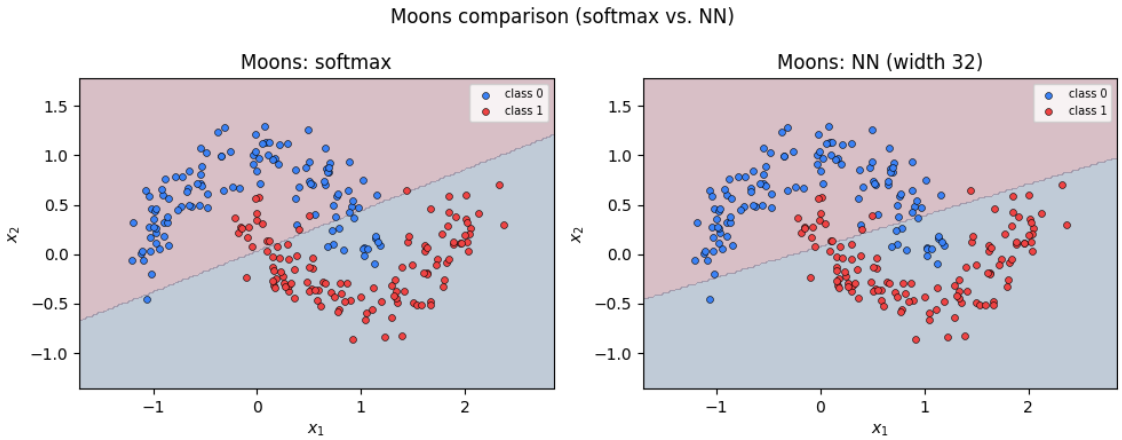

Figure 2: Decision boundaries on the moons task (left: softmax regression; right: neural network, $m = 32$).

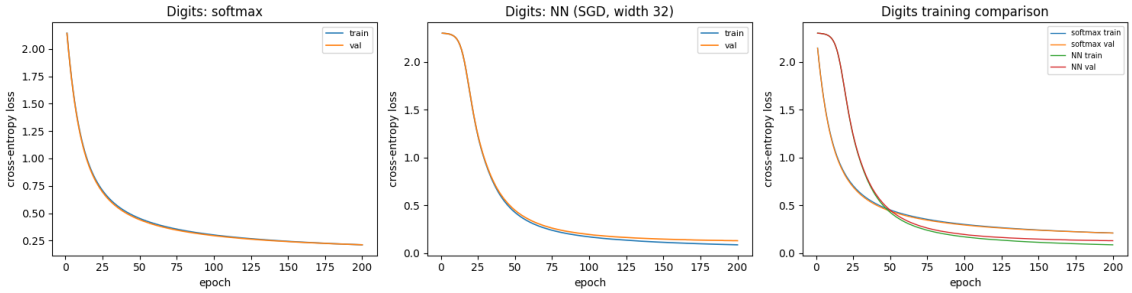

Figure 3: Training and validation cross-entropy curves on the digits benchmark (left: softmax regression; centre: neural network SGD; right: comparison).

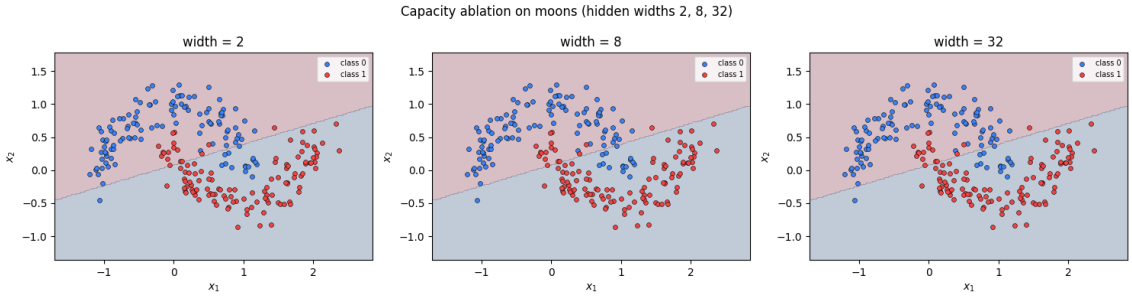

Figure 4: Decision boundaries for hidden widths $m \in \{2, 8, 32\}$ on the moons task.

4.5 Required optimizer study on digits

All three optimizers are compared under fixed defaults with the same split, preprocessing, epoch budget, and checkpoint rule. SGD makes steady but slow progress, still converging at epoch 200. Momentum accumulates gradient history and converges noticeably faster. Adam adapts per-parameter learning rates using first and second moment estimates, reaching the lowest validation loss in the fewest epochs (Table 5, Figure 5). Despite these differences in convergence speed, all three reach broadly similar final validation accuracy within the 200-epoch budget, suggesting the loss landscape is benign enough that optimizer choice affects training dynamics more than final solution quality.

4.6 Required failure-case analysis

Setup. The neural network was trained on the moons task with hidden width $m = 2$ and no L2 regularisation ($\lambda = 0$), and compared against $m = 32$ under identical conditions.

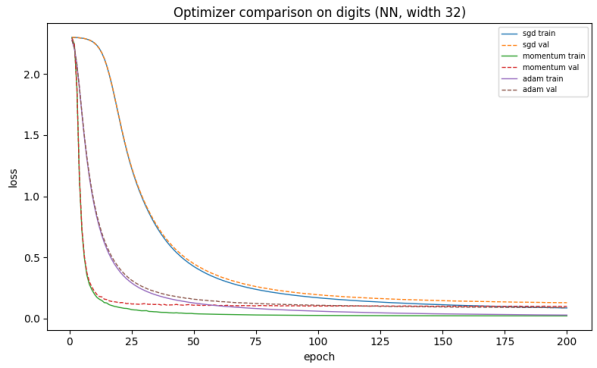

Figure 5: Training and validation cross-entropy curves for SGD, Momentum, and Adam on the digits benchmark (NN, $m = 32$).

Table 5: Optimizer comparison on the digits validation split (NN, $m = 32$, single seed).

Optimizer	Val Accuracy	Val Cross-Entropy	Best Epoch
SGD	0.9375	0.1929	200
Momentum	0.9429	0.1758	198
Adam	0.9565	0.1622	198

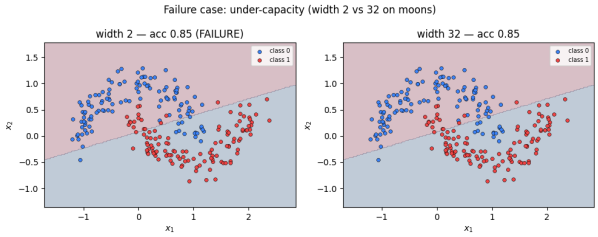

Figure 6: Failure case: width-2 network (left) vs. width-32 network (right) on the moons task. Both achieve 0.85 accuracy but the width-2 boundary is nearly linear.

Observation. The width-2 network achieves accuracy and cross-entropy comparable to softmax regression, offering no meaningful improvement over the linear baseline despite being a nonlinear model (Figure 6).

Mechanism. Each `tanh` unit partitions the 2D input space with a single oriented sigmoid-like fold. With only $m = 2$ units the model can produce at most two such folds, which is insufficient to carve out the curved boundary needed to separate the interleaving crescents. The hypothesis class is too small to represent the target geometry, and the network collapses to a nearly linear decision region. This is an **under-capacity failure**: the bottleneck is the size of the hypothesis class, not the optimizer or training budget.

Table 6: Test performance over 5 seeds (mean ± 95% CI) on the digits benchmark.

Model	Mean Acc.	95% CI	Mean CE	95% CI
Softmax regression	0.9386	± 0.0018	0.2693	± 0.0004
Neural network	0.9397	± 0.0050	0.1933	± 0.0103

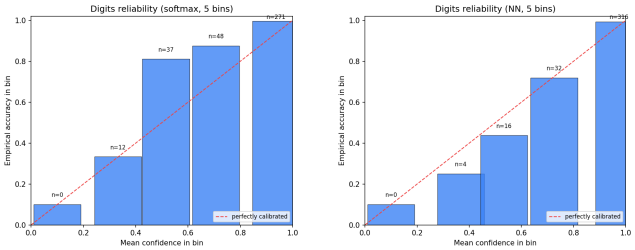

Figure 7: 5-bin reliability plots for softmax regression (left) and the neural network (right). The dashed line indicates perfect calibration.

4.7 Digits Benchmark: Repeated-Seed Evaluation

Final configurations selected using validation cross-entropy:

- Softmax regression: SGD, $\eta = 0.05$, $\lambda = 10^{-4}$.
- Neural network: SGD, $m = 32$, $\lambda = 10^{-4}$.

The accuracy confidence intervals overlap, meaning no statistically significant accuracy advantage can be claimed for either model at the 95% level. The variability across seeds is comparable to the inter-model gap, confirming that both models perform similarly on this benchmark under the fixed protocol. The neural network does, however, achieve consistently lower mean cross-entropy (0.1933 vs. 0.2693) with non-overlapping cross-entropy CIs, indicating better-calibrated predicted probabilities even where raw accuracy is indistinguishable.

5 Advanced Track: Prediction Confidence and Reliability (Track B)

This track analyses how well each model's predicted probabilities reflect true correctness on the digits test set. Confidence is defined as the maximum predicted class probability $\mathrm{conf}\mathbf{x} = \max_j p_j\mathbf{x}$, and predictive entropy as $H\mathbf{x} = -\sum_{j=1}^{k} p_j\mathbf{x} \log p_j\mathbf{x}$. A reliable model assigns high confidence and low entropy to correctly classified examples, and lower confidence and higher entropy to errors.

The test set is partitioned into 5 equal-width confidence bins and empirical accuracy is plotted against mean confidence per bin (Figure 7). A perfectly calibrated model lies on the diagonal. Figure 8 compares mean predictive entropy for correct versus incorrect predictions under both models.

Both models concentrate most examples in high-confidence bins and show the expected pattern: incorrectly classified examples carry higher entropy than correctly classified ones. The neural network's entropy on correct predictions (0.190) is

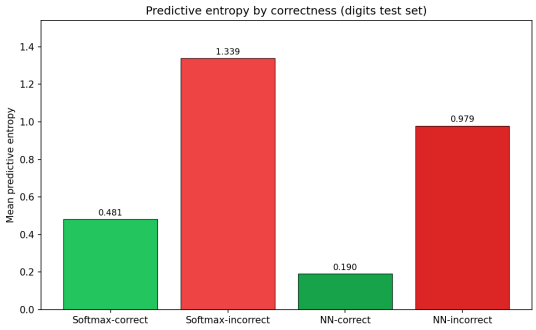

Figure 8: Mean predictive entropy for correct and incorrect predictions on the digits test set. Softmax-correct: 0.481, Softmax-incorrect: 1.339, NN-correct: 0.190, NN-incorrect: 0.979.

substantially lower than softmax regression's (0.481), indicating more concentrated and decisive probability distributions on examples it gets right. On incorrect predictions both models show high entropy (1.339 and 0.979 respectively), meaning errors tend to occur when the model is genuinely uncertain rather than overconfidently wrong. This is consistent with the neural network's lower mean cross-entropy (0.1933 vs. 0.2693) observed in the repeated-seed evaluation: the hidden layer improves not only the sharpness of correct predictions but also the overall calibration of the probability output. These findings tie back to the main project question — the hidden layer's representational advantage on the digits benchmark is more visible in probabilistic quality than in raw accuracy.

6 Discussion

The experiments collectively answer the central question: a one-hidden-layer network earns its added complexity only when the task geometry requires nonlinear representation.

On the linear Gaussian task both models achieve identical accuracy (0.95) and nearly identical cross-entropy. The Bayes-optimal boundary for two Gaussians with shared covariance is a hyperplane, which softmax regression represents exactly. The neural network finds no measurable improvement, confirming that additional capacity is unnecessary when the task is geometrically linear.

On the moons task the hidden layer provides a clear cross-entropy advantage (0.2695 vs. 0.2852) even where raw accuracy appears equal. The tanh feature map re-embeds the 2D input into a 32-dimensional representation in which the two crescents become more linearly separable. The capacity ablation reinforces this: with $m = 2$ the network collapses to a near-linear boundary and offers no gain over softmax regression, while $m = 32$ tracks the curved class structure closely. The bottleneck is representational, not computational.

On the digits benchmark the picture is more nuanced. Accuracy confidence intervals overlap across five seeds, so no statistically significant accuracy advantage can be claimed. The neural network does, however, achieve consistently lower cross-entropy (0.1933 vs. 0.2693) with non-overlapping CIs, suggesting better-calibrated probability estimates. The Track B analysis supports this directly: the neural network assigns lower entropy to correct predictions (0.190 vs. 0.481) and higher entropy to errors, indicating that its uncertainty estimates are more informative. The optimizer study shows that SGD, Momentum, and Adam all reach comparable final accuracy within 200 epochs, so optimizer choice affects training speed more than solution quality on this task.

The failure case teaches a precise lesson: under-capacity is a representational failure, not an optimization failure. With $m = 2$ the network cannot carve the curved moons boundary regardless of training budget. Capacity is therefore a design choice with geometric consequences, not a free parameter to tune blindly.

Repeated-seed evaluation adds an essential epistemic layer. Single-run results appeared to favour the neural network on accuracy, but the five-seed summary reveals that the gap is within seed-to-seed variability. Without repeated evaluation this uncertainty would be invisible, and a single run could lead to an overconfident conclusion.

7 Limitations

Several boundaries on these conclusions should be stated explicitly.

First, all experiments use a single hidden layer with `tanh` activation. The findings do not generalise to deeper architectures, alternative activations such as ReLU, or residual connections. The representational argument made here is specific to the one-hidden-layer setting studied.

Second, the digits benchmark is small (1,797 examples, 64 dimensions) and already well-structured. The advantage of the neural network may differ on larger, noisier, or higher-dimensional datasets. No claim is made about behaviour beyond this benchmark.

Third, five seeds provide only a coarse statistical summary. The 95% confidence intervals are wide relative to the inter-model accuracy gap, so the repeated-seed evaluation rules out a large advantage but cannot resolve small differences with confidence.

Fourth, the capacity ablation on moons uses $\lambda = 0$. Results may differ under nonzero regularisation, where the interaction between width and weight decay could shift the threshold at which capacity becomes the binding constraint.

Fifth, the Track B reliability analysis is descriptive. No formal calibration metric such as expected calibration error is computed and no recalibration method is applied. The conclusions about confidence and entropy are qualitative and should not be interpreted as a rigorous uncertainty quantification study.

Acknowledgments

We thank the instructors and reviewers of the Math4AI capstone for guidance and feedback.

References

[1] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. 2020. *Mathematics for Machine Learning*. Cambridge University Press.
[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.
[3] Kevin P. Murphy. 2022. *Probabilistic Machine Learning: An Introduction*. MIT Press.
[4] Andrew Ng and Kian Katanforoosh. 2024. CS229 Lecture Notes: Deep Learning. Stanford University lecture notes.
[5] Stanford CS231n Staff. 2024. Neural Networks Part 1: Setting up the Architecture. Course notes.
[6] Stanford CS231n Staff. 2024. Optimization Part 2: Backpropagation, Intuitions. Course notes.

A Appendix Material

A.1 Seed-by-Seed Results

Table 7 reports the per-seed test results from which the means and confidence intervals in Table 6 were computed. Softmax regression shows very low seed-to-seed variability (accuracy std = 0.0015), reflecting the convexity of its loss landscape. The neural network shows higher variability (accuracy std = 0.0040), expected given sensitivity to random initialisation.

A.2 Complete Numerical Results

Table 8 consolidates all test results across all datasets and models reported in the main body.

A.3 Individual Optimizer Training Curves

Figures 9–11 show individual training and validation curves for each optimizer, supplementing the combined comparison in Figure 5.

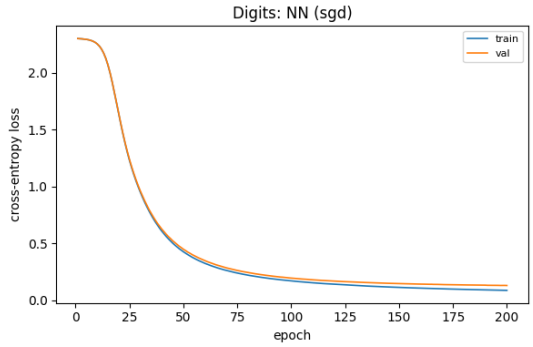

Figure 9: Training and validation curves for SGD ($\eta = 0.05$).

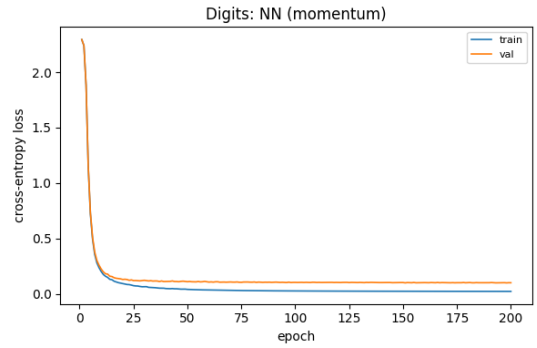

Figure 10: Training and validation curves for Momentum ($\eta = 0.05$, $\beta = 0.9$).

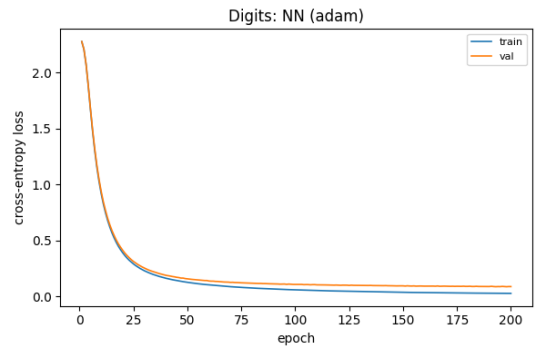

Figure 11: Training and validation curves for Adam ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$).

Contribution Statement

Aisel Mamedova implemented all implementation sanity checks (gradient verification, probability sum checks, tiny-subset overfitting, NaN/Inf guards) and wrote all four required mathematical derivations (NLL, backpropagation concept, full gradient derivation, geometric interpretation).

Shamistan Huseynov implemented the one-hidden-layer neural network from scratch including all three optimizers (SGD, Momentum, Adam) and completed the Track B confidence and reliability analysis.

Rahima Karimova implemented shared code infrastructure (data loader, metrics, plotting utilities), softmax regression, and ran all required ablations and repeated-seed evaluation on the digits benchmark.

Gulnur Mammadova ran all core experiments on all three datasets and wrote the full report.

All team members contributed to project setup, repository organisation, and final submission preparation.

Table 7: Per-seed test results on the digits benchmark.

Model	Metric	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Mean	Std
Softmax	Accuracy	0.9402	0.9375	0.9402	0.9375	0.9375	0.9386	0.0015
Softmax	Cross-entropy	0.2696	0.2695	0.2695	0.2687	0.2693	0.2693	0.0004
NN (SGD)	Accuracy	0.9375	0.9402	0.9402	0.9348	0.9457	0.9397	0.0040
NN (SGD)	Cross-entropy	0.1929	0.1885	0.2056	0.1958	0.1837	0.1933	0.0083

Table 8: Complete test results across all datasets and models.

Dataset	Model	Test Accuracy	Test Cross-Entropy
Gaussian	Softmax regression	0.9500	0.1539
Gaussian	Neural network ($m = 32$)	0.9500	0.1555
Moons	Softmax regression	0.8500	0.2852
Moons	Neural network ($m = 2$)	0.8500	0.2781
Moons	Neural network ($m = 8$)	0.8500	0.2712
Moons	Neural network ($m = 32$)	0.8500	0.2695
Digits	Softmax regression	0.9402	0.2696
Digits	Neural network (SGD)	0.9375	0.1929
Digits	Neural network (Momentum)	0.9429	0.1758
Digits	Neural network (Adam)	0.9565	0.1622